

Local execution, sandbox, and why hooks can save tokens

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 6 of 12 · English notes

The token argument in session 6 is quantitative, not aesthetic. Asking Claude to re-emit a file already written so that quotes and line breaks match house style is a second generation pass. Running Prettier (or any formatter/linter) in the **sandbox** after Write is local CPU. The trainer's order-of-magnitude for a full-fledged generation: on the order of **500–2000 tokens saved** by not making the model reprocess formatting. Hooks are the mechanism; the sandbox is where those commands actually run.

SANDBOX IS PRODUCT-MANAGED

You do not bring your own sandbox. The trainer: Claude manages it; there is no custom sandbox. User-scoped plugin installs land in user space; project-scoped installs in project sandboxes. Paths differ on Mac vs Windows and were not fully resolved live — trust current **Claude Code** rather than a remembered home directory.

What “local” means here

Echo, NPX, and shell scripts execute on the machine/workspace where Claude Code is running, inside the same isolated environment that already held MCP temp files and generated trees (session 4's Docker-like analogy). That isolation is why formatters can touch the working copy without sending the whole file back through the model. It is also why a hook that works in the terminal session may not be visible in the IDE webview until you reload window / reload webviews (Ctrl+Shift+P style) or enable the preview hook setting.

Where tokens still get spent

- CLAUDE.md and skills are still appended to model requests — that is billed context, even if you moved Prettier out.
- Verify/clean-code loops (Module 04–05) are billed; disable in memory if you will test locally.
- Attaching a whole backend folder instead of `openapi.yaml` is billed.
- JPEG/SVG diagrams instead of Mermaid are billed (session 4).
- MCP tool results occupy context while they sit in the window, even though the fetch itself is a tool call.
- A hook that *prints a novel into the prompt* on every tool use would be a bad hook — keep hook output short.

Scripts as the body of local policy

Hooks are not limited to `npx prettier`. Anything a shell script can do is in scope: Python, other binaries, `protect-files.sh`, `block-dangerous-commands.sh`. The trainer kept scripts under a `hooks/` directory in the working tree and referenced them from `settings.json`. Classmates wanted the comment headers of those scripts so they could copy the intent without reverse-engineering — those files

were pushed to the public demo repo. Comments in the script are for humans and for you; they do not need to live in the LLM prompt.

Local command	Typical hook event	Token effect
npx prettier / eslint --fix	post-tool-use on Edit/Write	Avoids a format regeneration pass (~hundreds-thousands of tokens on a large write).
protect-files.sh (deny .env, package-lock.json, .git, Prisma secrets, ...)	pre-tool-use on Edit/Write	Prevents secret-file rewrites; also avoids messy “undo this” generations.
block-dangerous-commands.sh (rm, reckless git, ...)	pre-tool-use on Bash	Stops auto-mode disasters; cheaper than repairing a deleted tree with the model.
echo status line	post-tool-use	Negligible; observability only.

Plugins still unpack into the sandbox

When you install a marketplace or custom plugin at project scope, settings.json gains a reference; Claude downloads the bundle and makes skills/agents/hooks available from the sandbox. You may not see new folders in the git working tree. User-level install goes to user space instead. Treat plugins as a library you consume, not as files you casually edit unless you are a collaborator on the source repo (GitHub).

Exam percentages the trainer attached to this era of the course

Still on 13 August, after hooks, he mapped Foundation weights (confirm against Skilljar before you sit): agent architecture and operations ~27%, tool design and MCP ~18%, Claude Code configuration and workflows ~20%, prompt engineering and structured output ~20%, context management and reliability ~15%. Hooks, sandbox, and token-aware local tools sit in configuration/workflows and context/reliability. Do not quote the percentages as eternal; quote the idea that local deterministic work should be stripped out of the billed loop.

Study check

If a scenario asks how to cut cost on a greenfield Claude Code repo that already has a strict style guide, the high-scoring answer is: keep architectural rules in CLAUDE.md; move formatter and file-guard scripts to hooks running in the sandbox; do not send the formatter spec through the model on every Write.

Official documentation

- **Claude Code overview**
<https://code.claude.com/docs/en/overview>
Sandbox, hooks, and plugin install scopes.
- **Rate limits**
<https://docs.claude.com/en/api/rate-limits>
Usage meters make the 500–2000 token story operational, not academic.
- **Prompt caching**

<https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Later API feature; not a substitute for local hooks in Claude Code.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Org usage and model access still bound the remaining billed tokens.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.